Practical Assignment #1 Report

Network Configuration with IPTables and Suricata

Miguel Silva Guilherme Rodrigues

March 2026

Contents

1 Introduction **3**

2 Scenario **3**

3 Configuration of the Solution **4**
3.1 Network Variables 4
3.2 Interface Setup 4
3.3 Firewall Service Disablement and Initial IPTables Cleanup 5

4 Firewall Configuration **6**
4.1 Protection of the Router 6
4.2 Direct Communications Without NAT 6
4.2.1 DNS 6
4.2.2 SMTP 7
4.2.3 POP and IMAP 7
4.2.4 HTTP and HTTPS 7
4.2.5 OpenVPN 7

5 NAT Configuration **8**
5.1 Inbound NAT (Port Forwarding) 8
5.1.1 FTP to the Internal FTP Server 8
5.1.2 SSH to the Datastore Server 8
5.2 Outbound NAT 9

6 Suricata Configuration **9**

7 Attacks Considered and Detection with Suricata **10**
7.1 Port Scanning 10
7.2 SQL Injection 10
7.3 Cross-Site Scripting (XSS) 10

8 Validation Tests **11**
8.1 Router Protection 11
8.2 DMZ Services Access Control 13
8.3 NAT Configuration Validation 15
8.4 Suricata IDS/IPS 16

9 Sources **16**

1 Introduction

The goal of this assignment is to implement a firewall capable of controlling communications between an internal network, a DMZ, and the Internet, while also detecting and blocking malicious activity. The solution must protect the router itself, allow only the services required in the assignment, support NAT for specific internal services, and integrate Suricata to detect attacks such as port scanning, SQL injection, and cross-site scripting.

This report is divided into three main parts. First, it describes the scenario and the configurations used to implement the solution. Second, it explains the attacks considered and how Suricata detects and blocks them. Finally, it presents the tests performed to validate the complete solution.

2 Scenario

The scenario considered in this assignment is composed of three different network zones: the Internet, the DMZ, and the internal network. The Internet represents the untrusted external network. The DMZ contains the public-facing services that must be accessible from outside, such as DNS, SMTP, mail, web, and VPN gateway services. The internal network contains protected hosts and services that should not be directly exposed.

The router/firewall is the central element interconnecting these three zones. It is responsible for routing packets between them, applying packet filtering rules, performing NAT, and sending traffic to Suricata for inspection.

The configuration of each host is performed using the corresponding shell script file. In particular:

- the Internet host is configured with `internet_host_config.sh` file;
- the DMZ host is configured with `dmz_host_config.sh` file;
- the internal host is configured with `internal_host_config.sh` file;
- the router/firewall is configured with `router_host_config.sh` file.

This organization makes the deployment clearer and easier to reproduce, since each machine has an explicit configuration script associated with its role in the scenario.

3 Configuration of the Solution

3.1 Network Variables

The router script begins by defining variables for interfaces, networks, and hosts. This has been updated to reflect the exact subnets and assigned IP addresses for each segment:

```
INTERNET_IF="enp0s10"
INTERNET_NET="193.136.0.0/15"
INTERNET_IP="193.136.0.254"
INTERNET_EDEN="193.136.212.1"
INTERNET_DNS2="193.137.16.75"

INTERNAL_IF="enp0s9"
INTERNAL_NET="192.168.10.0/24"
INTERNAL_IP="192.168.10.254"
INTERNAL_FTP_SERVER="192.168.10.2"
INTERNAL_DATASTORE_SERVER="192.168.10.3"

DMZ_IF="enp0s8"
DMZ_NET="23.214.219.128/25"
DMZ_IP="23.214.219.254"
DMZ_SMTP_SERVER="23.214.219.129"
DMZ_DNS1_SERVER="23.214.219.130"
DMZ_MAIL_SERVER="23.214.219.131"
DMZ_WWW_SERVER="23.214.219.132"
DMZ_VPN_GW_SERVER="23.214.219.133"
```

These variables make the configuration easier to read and update, since all addresses and host roles are defined centrally.

3.2 Interface Setup

The firewall/router is configured with three interfaces, one for each network segment:

```
ifconfig $DMZ_IF      $DMZ_IP      netmask 255.255.255.128 up
ifconfig $INTERNAL_IF $INTERNAL_IP netmask 255.255.255.0 up
ifconfig $INTERNET_IF $INTERNET_IP netmask 255.254.0.0 up
```

IP forwarding and FTP connection tracking modules are then enabled so that the system can operate as a router:

```
modprobe nf_conntrack_ftp
modprobe nf_nat_ftp
echo 1 > /proc/sys/net/ipv4/ip_forward
sysctl -w net.netfilter.nf_conntrack_helper=1
```

3.3 Firewall Service Disablement and Initial IPTables Cleanup

To ensure IPTables operates without interference, the default `firewalld` service is stopped and disabled. Before applying new rules, all previous rules and chains are flushed (including the `nat` and `mangle` tables). Strict default drop policies are applied, loopback traffic is permitted, and existing stateful connections are globally accepted to streamline the ruleset:

```
# Disable firewall
systemctl stop firewalld
systemctl disable firewalld

# Iptables Clear
iptables -F
iptables -X
iptables -t nat -F
iptables -t nat -X
iptables -t mangle -F
iptables -t mangle -X

# Drop everything by default
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT DROP

# loopback
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT

iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
```

4 Firewall Configuration

The firewall policy relies heavily on strict stateful inspection, restricting rules specifically to `NEW` connections.

4.1 Protection of the Router

The router itself is protected by only allowing the minimum required traffic. DNS traffic generated by the router is allowed with:

```
iptables -A OUTPUT -o $INTERNET_IF -p udp --dport 53 -m state --state NEW -j
    ACCEPT
iptables -A OUTPUT -o $INTERNET_IF -p tcp --dport 53 -m state --state NEW -j
    ACCEPT
```

SSH access to the router is restricted to the internal network and the VPN gateway:

```
iptables -A INPUT -i $INTERNAL_IF -s $INTERNAL_NET -p tcp --dport 22 -m state --
    state NEW -j ACCEPT
iptables -A INPUT -i $DMZ_IF -s $DMZ_VPN_GW_SERVER -p tcp --dport 22 -m state --
    state NEW -j ACCEPT
```

4.2 Direct Communications Without NAT

The assignment requires some services to be accessed directly, without NAT.

4.2.1 DNS

DNS access to the local DMZ DNS server is allowed with:

```
iptables -A FORWARD -d $DMZ_DNS1_SERVER -p udp --dport 53 -m state --state NEW -
    j ACCEPT
iptables -A FORWARD -d $DMZ_DNS1_SERVER -p tcp --dport 53 -m state --state NEW -
    j ACCEPT
```

The local DNS server is authorized to resolve names using the Internet:

```
iptables -A FORWARD -i $DMZ_IF -o $INTERNET_IF -s $DMZ_DNS1_SERVER -p udp --
    dport 53 -m state --state NEW -j ACCEPT
iptables -A FORWARD -i $DMZ_IF -o $INTERNET_IF -s $DMZ_DNS1_SERVER -p tcp --
    dport 53 -m state --state NEW -j ACCEPT
```

DNS zone synchronization between the DMZ DNS server and the external DNS server is allowed with:

```
iptables -A FORWARD -i $DMZ_IF -o $INTERNET_IF -s $DMZ_DNS1_SERVER -d
    $INTERNET_DNS2 -p tcp --dport 53 -m state --state NEW -j ACCEPT
iptables -A FORWARD -i $INTERNET_IF -o $DMZ_IF -s $INTERNET_DNS2 -d
    $DMZ_DNS1_SERVER -p tcp --dport 53 -m state --state NEW -j ACCEPT
```

4.2.2 SMTP

SMTP traffic to the SMTP server is permitted by:

```
iptables -A FORWARD -d $DMZ_SMTP_SERVER -p tcp --dport 25 -m state --state NEW -
    j ACCEPT
```

4.2.3 POP and IMAP

Mail retrieval through POP and IMAP is allowed with:

```
iptables -A FORWARD -d $DMZ_MAIL_SERVER -p tcp -m multiport --dports
    110,143,993,995 -m state --state NEW -j ACCEPT
```

4.2.4 HTTP and HTTPS

Web access to the public web server is allowed through:

```
iptables -A FORWARD -d $DMZ_WWW_SERVER -p tcp -m multiport --dports 80,443 -m
    state --state NEW -j ACCEPT
```

4.2.5 OpenVPN

OpenVPN access to the VPN gateway is allowed with:

```
iptables -A FORWARD -d $DMZ_VPN_GW_SERVER -p udp --dport 1194 -m state --state
    NEW -j ACCEPT
```

VPN clients are then allowed to reach the internal network using:

```
iptables -A FORWARD -i $DMZ_IF -o $INTERNAL_IF -s $DMZ_VPN_GW_SERVER -m state --
    state NEW -j ACCEPT
```

5 NAT Configuration

5.1 Inbound NAT (Port Forwarding)

The external IP of the firewall is used to publish selected internal services.

5.1.1 FTP to the Internal FTP Server

FTP access to the internal FTP server is enabled through NAT for both active and passive modes:

```
PASV_MIN=30000
PASV_MAX=30050

iptables -t nat -A PREROUTING -i $INTERNET_IF -d $INTERNET_IP -p tcp --dport 21
    -j DNAT --to-destination $INTERNAL_FTP_SERVER
iptables -t nat -A PREROUTING -i $INTERNET_IF -d $INTERNET_IP -p tcp --dport ${
    PASV_MIN}:${PASV_MAX} -j DNAT --to-destination $INTERNAL_FTP_SERVER

iptables -A FORWARD -i $INTERNET_IF -o $INTERNAL_IF -d $INTERNAL_FTP_SERVER -p
    tcp --dport 21 -m state --state NEW -j ACCEPT
iptables -A FORWARD -i $INTERNET_IF -o $INTERNAL_IF -d $INTERNAL_FTP_SERVER -p
    tcp --dport 20 -m state --state NEW -j ACCEPT
iptables -A FORWARD -i $INTERNET_IF -o $INTERNAL_IF -d $INTERNAL_FTP_SERVER -p
    tcp --dport ${PASV_MIN}:${PASV_MAX} -m state --state NEW -j ACCEPT
```

5.1.2 SSH to the Datastore Server

SSH connections are redirected to the internal datastore server, but only if they originate from `eden` or `dns2`:

```
iptables -t nat -A PREROUTING -i $INTERNET_IF -s $INTERNET_EDEN -d $INTERNET_IP
    -p tcp --dport 22 -j DNAT --to-destination $INTERNAL_DATASTORE_SERVER
iptables -t nat -A PREROUTING -i $INTERNET_IF -s $INTERNET_DNS2 -d $INTERNET_IP
    -p tcp --dport 22 -j DNAT --to-destination $INTERNAL_DATASTORE_SERVER

iptables -A FORWARD -i $INTERNET_IF -o $INTERNAL_IF -s $INTERNET_EDEN -d
    $INTERNAL_DATASTORE_SERVER -p tcp --dport 22 -m state --state NEW -j ACCEPT
iptables -A FORWARD -i $INTERNET_IF -o $INTERNAL_IF -s $INTERNET_DNS2 -d
    $INTERNAL_DATASTORE_SERVER -p tcp --dport 22 -m state --state NEW -j ACCEPT
```

5.2 Outbound NAT

The internal network is allowed to access Internet/DMZ services through NAT using SNAT.

```
iptables -t nat -A POSTROUTING -s $INTERNAL_NET -o $INTERNET_IF -j SNAT --to-
    source $INTERNET_IP
```

Forwarding limits for the internal network are strictly enforced for DNS, HTTP, HTTPS, SSH, and FTP:

```
### Domain name resolutions using DNS
iptables -A FORWARD -i $INTERNAL_IF -o $INTERNET_IF -s $INTERNAL_NET -p udp --
    dport 53 -m state --state NEW -j ACCEPT
iptables -A FORWARD -i $INTERNAL_IF -o $INTERNET_IF -s $INTERNAL_NET -p tcp --
    dport 53 -m state --state NEW -j ACCEPT
### HTTP, HTTPS and SSH connections
iptables -A FORWARD -i $INTERNAL_IF -o $INTERNET_IF -s $INTERNAL_NET -p tcp -m
    multiport --dports 80,443,22 -m state --state NEW -j ACCEPT
### FTP connections (in passive and active modes) to external FTP servers
iptables -A FORWARD -i $INTERNAL_IF -o $INTERNET_IF -s $INTERNAL_NET -p tcp --
    dport 21 -m state --state NEW -j ACCEPT
iptables -A FORWARD -i $INTERNAL_IF -o $INTERNET_IF -s $INTERNAL_NET -p tcp --
    dport 20 -m state --state NEW -j ACCEPT
```

6 Suricata Configuration

Suricata is integrated with IPTables using NFQUEUE. The insertion rules (`-I 1`) ensure that packets are queued for inspection before they hit the default ACCEPT rules:

```
iptables -I INPUT 1 -j NFQUEUE --queue-num 0
iptables -I OUTPUT 1 -j NFQUEUE --queue-num 0
iptables -I FORWARD 1 -j NFQUEUE --queue-num 0
```

The Suricata configuration files are copied into place:

```
cp suricata.yaml /etc/suricata/suricata.yaml
cp local.rules /var/lib/suricata/rules/local.rules
```

Finally, Suricata is started in IDS mode with NFQUEUE as the capture method, and with the highest level of verbosity:

```
suricata -c /etc/suricata/suricata.yaml -vvvv -q 0
```

7 Attacks Considered and Detection with Suricata

7.1 Port Scanning

Port scanning is used by attackers to discover open ports and running services on a target system. A scanner sends packets to multiple ports and observes the responses to identify which services are available.

Suricata can detect port scanning by identifying repeated connection attempts from the same source to multiple ports in a short period of time. Once this behaviour matches a configured rule, Suricata generates an alert and the packet can be blocked through NFQUEUE processing.

7.2 SQL Injection

SQL injection is an attack in which malicious SQL code is inserted into application input fields. If the application does not validate the input correctly, the attacker may read or manipulate data in the database.

In this solution, Suricata inspects application-layer payloads, especially HTTP requests, using both the ET rules and the local rules. It looks for suspicious patterns such as `' OR 1=1`, `UNION SELECT`, comment markers, or encoded SQL fragments. When detected, the traffic is logged and blocked.

7.3 Cross-Site Scripting (XSS)

Cross-Site Scripting consists of injecting malicious JavaScript or HTML into a web application so that it is later executed in the victim's browser. This may be used to steal sessions, redirect users, or manipulate the displayed page.

In this solution, Suricata detects XSS attempts by looking for suspicious script tags, JavaScript fragments, inline event handlers, and encoded payloads commonly associated with XSS attacks. Matching traffic is alerted and blocked.

8 Validation Tests

To ensure the correct operation of both IPTables and Suricata, several tests were executed across the different network segments.

8.1 Router Protection

The router must be protected against unauthorized access while being able to perform necessary outbound requests, such as DNS resolution. (An additional network interface NAT was temporarily added in order to test the iptables rules.)

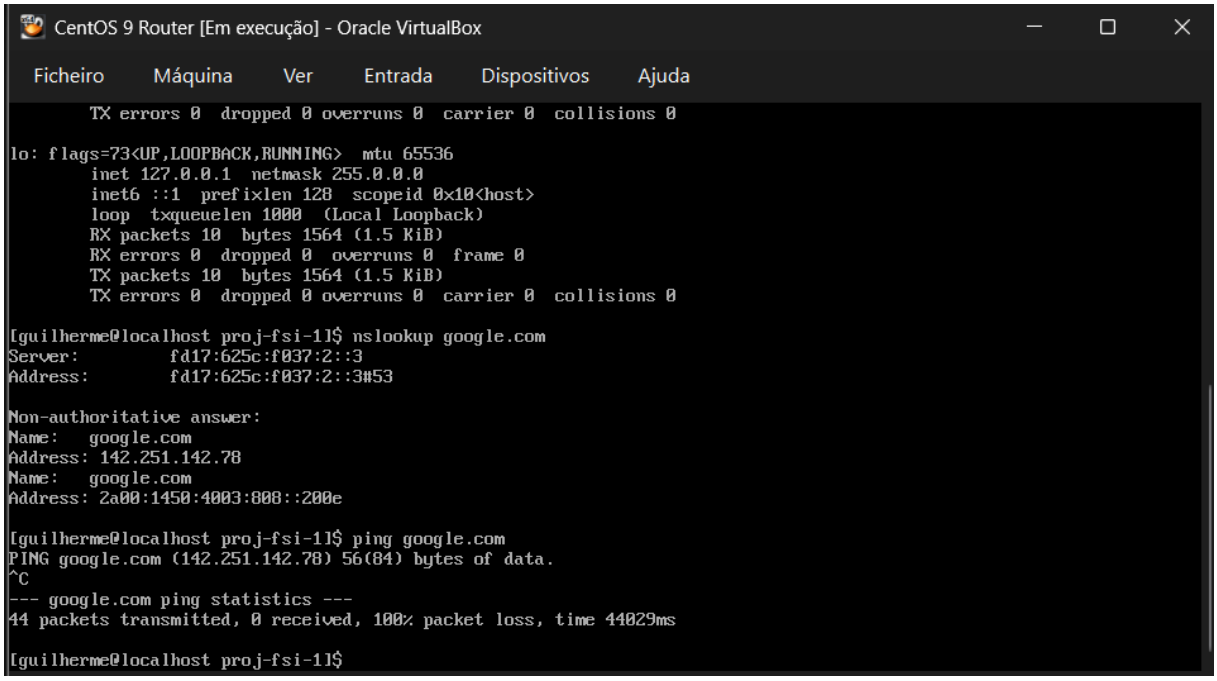


Figure 1: Router executing DNS resolution successfully and blocking outbound ICMP.

As shown in Figure 1, the router is able to resolve `google.com` using DNS (UDP port 53). However, the `ping` command results in 100% packet loss. This validates that only explicitly allowed outbound traffic is permitted by the router's `OUTPUT` chain.

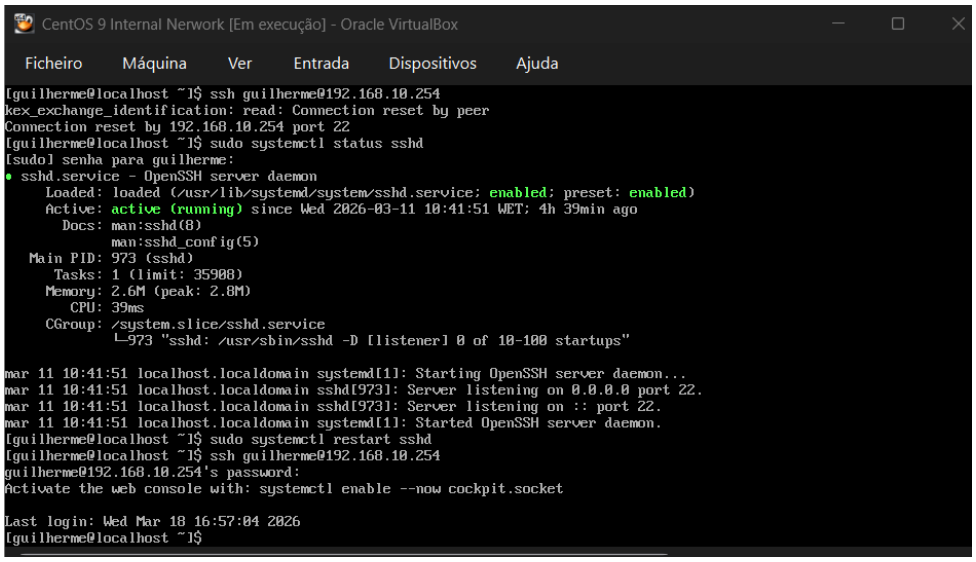


Figure 2: Successful SSH connection to the router from the Internal Network.

Figure 2 validates the rule that allows SSH access to the router solely from the Internal Network.

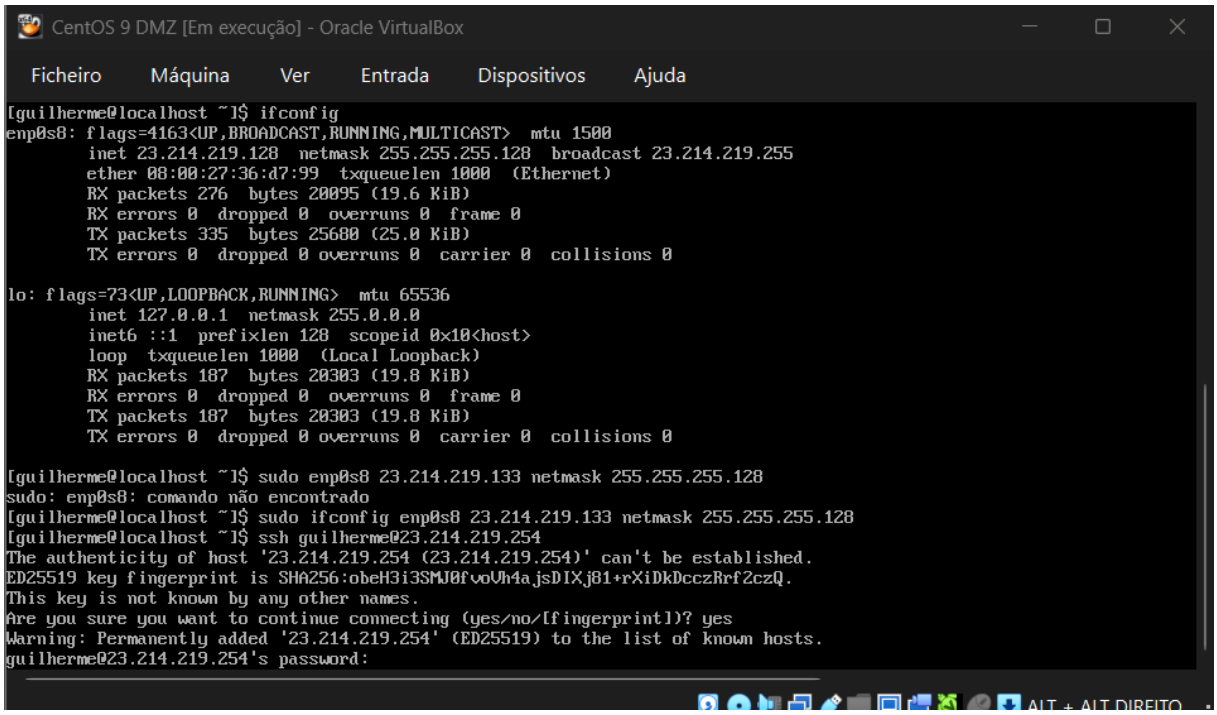


Figure 3: Successful SSH connection to the router from the DMZ VPN Gateway.

Similarly, Figure 3 demonstrates a successful SSH login to the router originating from the DMZ VPN Gateway (`23.214.219.133`), confirming the specific exemption rule applied for this host.

8.2 DMZ Services Access Control

Access to the services located in the DMZ must be strictly limited to the specified ports, blocking all other connection attempts.

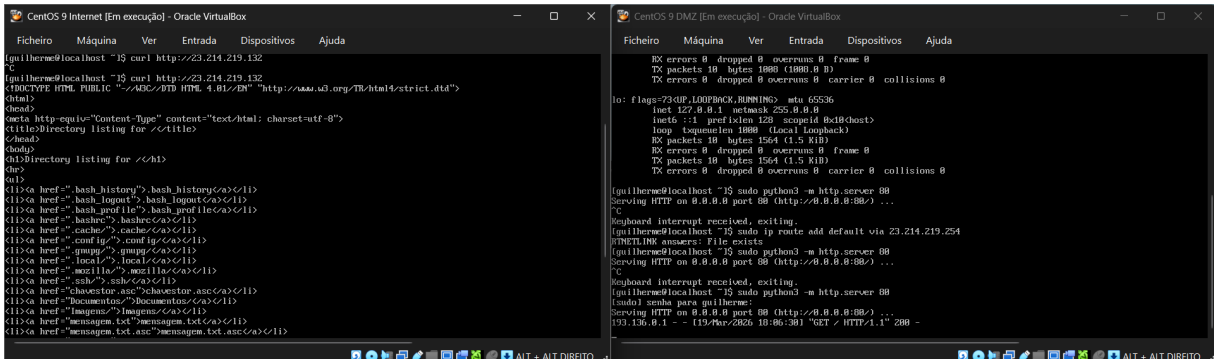


Figure 4: Successful HTTP request from the Internet to the DMZ WWW Server.

Figure 4 demonstrates the Internet host successfully executing an HTTP `curl` request to the DMZ WWW server (`23.214.219.132`). The right side shows the server receiving the request, confirming HTTP forwarding works.

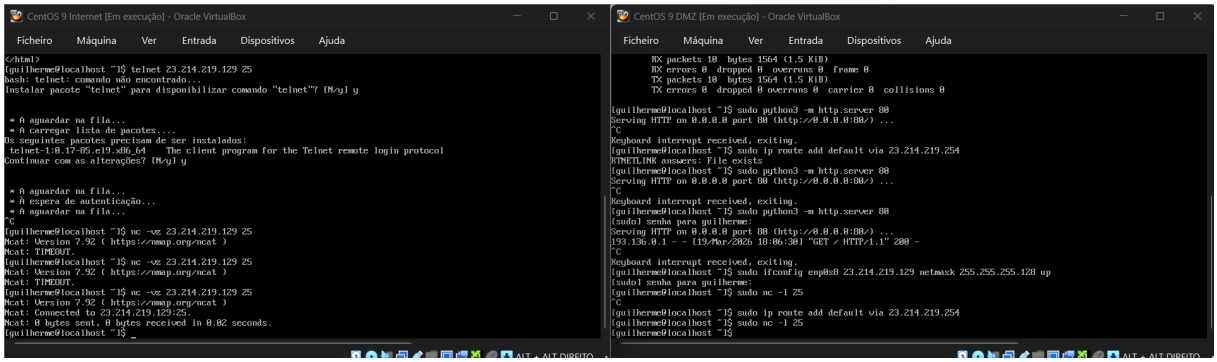


Figure 5: Successful SMTP connection from the Internet to the DMZ SMTP Server.

As seen in Figure 5, testing connectivity via `nc` to port 25 of the DMZ SMTP Server (`23.214.219.129`) returns a successful connection, validating the firewall's SMTP forward rules.

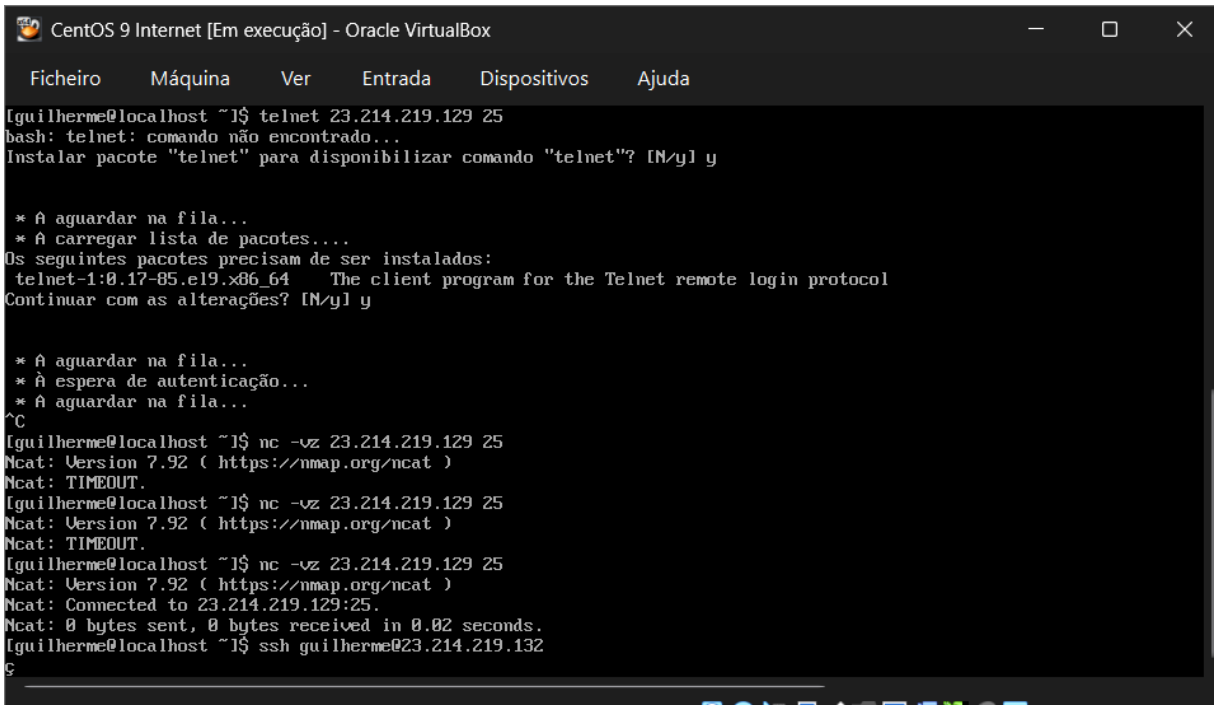
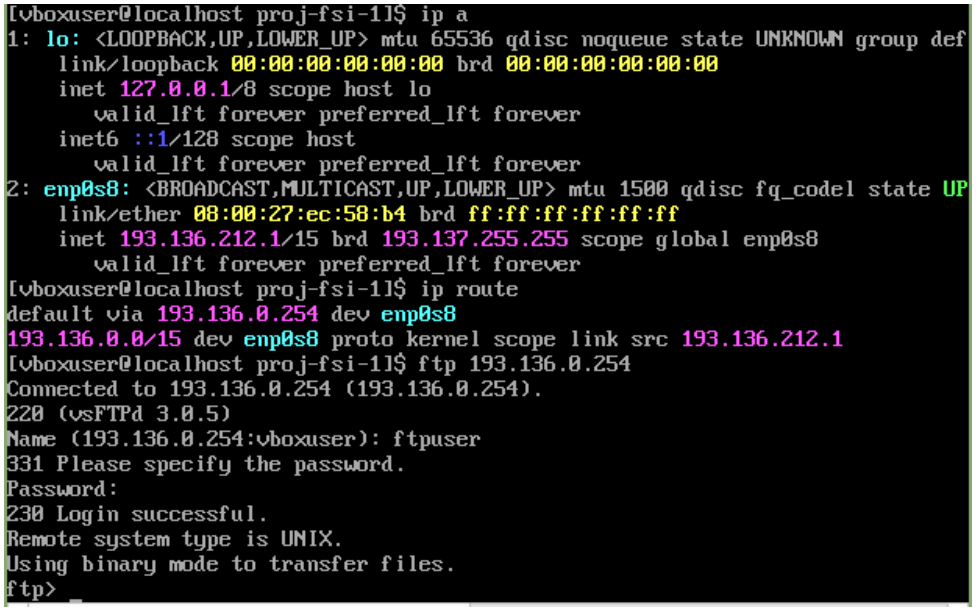


Figure 6: Blocked SSH connection attempt from the Internet to the DMZ WWW Server.

Conversely, Figure 6 shows an external host attempting an SSH connection to the DMZ WWW server. The connection hangs and times out, validating the default drop policy for unauthorized ports.

8.3 NAT Configuration Validation

To ensure the correct operation of Inbound NAT, external clients must be able to reach specific internal services using the router's public IP.



```
[vboxuser@localhost proj-fsi-1]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    link/ether 08:00:27:ec:58:b4 brd ff:ff:ff:ff:ff:ff
    inet 193.136.212.1/15 brd 193.137.255.255 scope global enp0s8
       valid_lft forever preferred_lft forever
[vboxuser@localhost proj-fsi-1]$ ip route
default via 193.136.0.254 dev enp0s8
193.136.0.0/15 dev enp0s8 proto kernel scope link src 193.136.212.1
[vboxuser@localhost proj-fsi-1]$ ftp 193.136.0.254
Connected to 193.136.0.254 (193.136.0.254).
220 (vsFTPd 3.0.5)
Name (193.136.0.254:vboxuser): ftpuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> _
```

Figure 7: Successful FTP access to the Internal Server via NAT from the Internet.

Figure 7 displays an external client (simulating the `eden` host) connecting to the router's Internet IP (`193.136.0.254`) on port 21. The connection is successfully DNAT to the Internal FTP server, authenticating the user and confirming both the port forwarding and active/passive data rules.

8.4 Suricata IDS/IPS

Suricata must be able to inspect traffic across the firewall and actively drop packets that match malicious signatures.

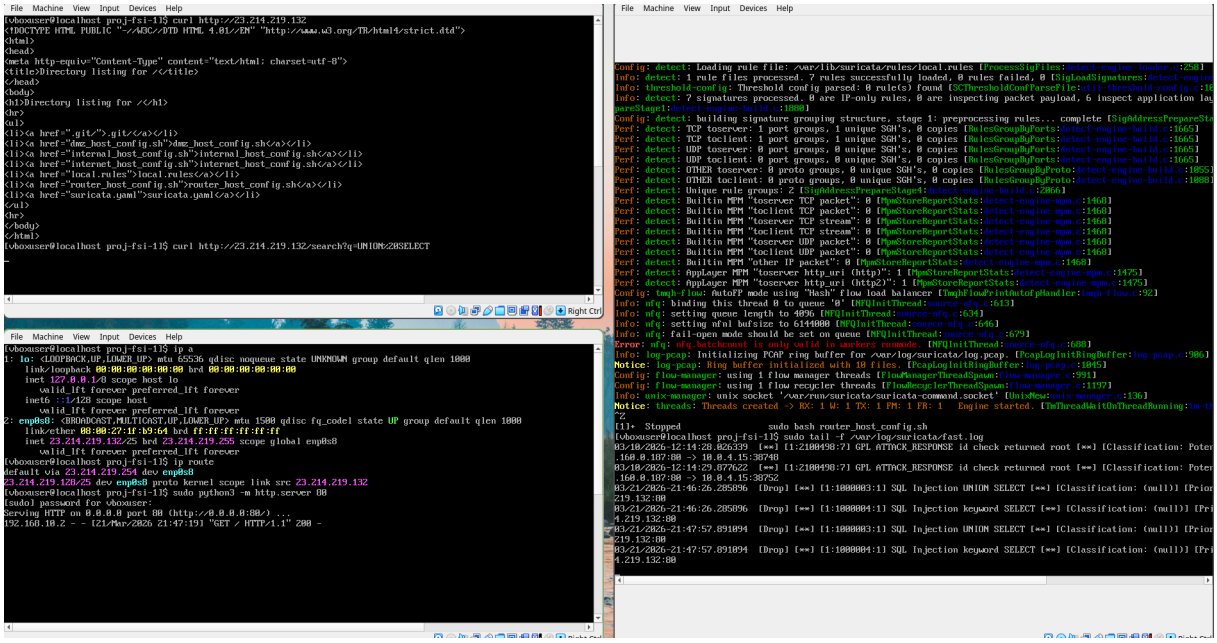


Figure 8: Suricata detecting and actively dropping an HTTP request containing a SQL Injection payload.

Figure 8 demonstrates an external client attempting to send an HTTP GET request containing a SQL injection payload (`UNION SELECT`). The right side displays Suricata's `fast.log`, which clearly shows the event being classified as a SQL Injection attack and the action taken as `[Drop]`. This validates that Suricata is operating in inline IPS mode via NFQUEUE, actively protecting the servers.

9 Sources

- **Gemini** — Google https://gemini.google.com/app
- **iptables(8) - Linux man page** https://linux.die.net/man/8/iptables
- **Suricata User Guide, Version 8.0.4** — OISF https://docs.suricata.io/en/suricata-8.0.4/